ERP FLOW STUDIOS

An Intelligent Clinic Management System

Source Code Submission

(First and Last Portions of Source Code)

Prepared for Copyright Submission

Authors:

Kushagra Juneja

Janvi Jain

Vishal Raj

Shikha Tuteja

File: pages/_app.tsx

```
import '../styles/globals.css'
import type { AppProps } from 'next/app'
import { SpeedInsights } from '@vercel/speed-insights/next'
import { Analytics } from '@vercel/analytics/next'
import { useRouter } from 'next/router'
import { useEffect, useRef, useState } from 'react'
import Head from 'next/head'
import Script from 'next/script'
import dynamic from 'next/dynamic'
import Layout from '../components/Layout'
import ToastNotification from '../components/ToastNotification'
import { useRelease } from '../hooks/useRelease'
import { useToast } from '../hooks/useToast'
import { useVersionCheck } from '../hooks/useVersionCheck'
import { ImportProvider } from '../contexts/ImportContext'
import { DataCacheProvider } from '../contexts/DataCacheContext'
import { AuthProvider, useAuth } from '../contexts/AuthContext'
import { DoctorProvider } from '../contexts/DoctorContext'
import { PageStateProvider } from '../contexts/PageStateContext'
import VoiceMicButton from '../components/VoiceMicButton'
import LanguageSwitcher from '../components/LanguageSwitcher'
import GoogleAnalytics from '../components/GoogleAnalytics'
import { initializeSyncQueue } from '../utils/syncQueue'
import { clearCachedCurrentUser, getCachedCurrentUser, setCachedCurrentUser } from
'../lib/currentUserStorage'
import { initializeMobileNotifications } from '../lib/mobileNotifications'
import { initializeAndroidPermissions } from '../lib/mobilePermissions'
import { SITE_DESCRIPTION, SITE_OG_DESCRIPTION, SITE_TITLE, buildCanonicalUrl, getSiteUrl,
normalizePath } from '../lib/seo'
import { OFFLINE_MODE_EVENT, getEffectiveOnlineState } from '../lib/offlineMode'
import '../utils/clearStorage' // Make storage utilities available in browser console

const VersionUpdateModal = dynamic(() => import('../components/VersionUpdateModal'))
const ReleaseBanner = dynamic(() => import('../components/ReleaseBanner'))
const UpdateModal = dynamic(() => import('../components/UpdateModal'))

const SITE_KEYWORDS = 'clinic management software, healthcare ERP, clinic ERP system,
hospital management software, medical practice management, ERP Flow Studios'
const SITE_URL = getSiteUrl()
const SITE_OG_IMAGE = `${SITE_URL}/og-image.png`

type UpdaterEventName =
  | 'checking-for-update'
  | 'update-available'
  | 'download-progress'
  | 'update-downloaded'
  | 'error'

interface UpdaterEventPayload {
  event: UpdaterEventName
  status?: string
  version?: string | null
  percent?: number
  error?: string
}

declare global {
  interface Window {
    electronAPI?: {
      quitApp?: () => void
```

```
      startUpdateDownload?: () => Promise<{ ok: boolean; error?: string }>
      checkForUpdates?: () => Promise<{ ok: boolean; error?: string }>
      onUpdaterEvent?: (callback: (payload: UpdaterEventPayload) => void) => () => void
    }
  }
}

/** Bridge that syncs _app.tsx's activeUser into AuthContext so pages can
 *  use `useAuth()` instead of fetching /api/auth/me themselves. Must be
 *  rendered INSIDE AuthProvider. */
function AuthSync({ activeUser }: { activeUser: any | null | undefined }) {
  const { setExternalUser } = useAuth()
  useEffect(() => {
    // Wait until _app resolves auth state; avoids transient null redirects.
    if (activeUser === undefined) return
    setExternalUser(activeUser)
  }, [activeUser, setExternalUser])
  return null
}

export default function App({ Component, pageProps }: AppProps) {
  const router = useRouter()
  const [authChecked, setAuthChecked] = useState(false)
  const [isTransitioning, setIsTransitioning] = useState(false)
  const [isNativeMobileApp, setIsNativeMobileApp] = useState(false)
  const [mobileSlideDirection, setMobileSlideDirection] = useState<'left' | 'right'>('left')
  const [faviconUrl, setFaviconUrl] = useState('/favicon.png')
  const [pageTitle, setPageTitle] = useState(SITE_TITLE)
  const [isProUser, setIsProUser] = useState(() => typeof window !== 'undefined' ?
localStorage.getItem('clinicIsPro') === '1' : false)
  const [activeUser, setActiveUser] = useState<any | null | undefined>(undefined)
  const [trialReminderVisible, setTrialReminderVisible] = useState(false)
  const [trialReminderMessage, setTrialReminderMessage] = useState('')
  const { toasts, removeToast, showError, showInfo, showSuccess } = useToast()
  const [isOnline, setIsOnline] = useState(true)
  const { hasUpdate, latestRelease, displayType, dismissRelease } = useRelease()
  const versionCheck = useVersionCheck()
  const [updateDismissed, setUpdateDismissed] = useState(false)
  const [hasDesktopUpdaterBridge, setHasDesktopUpdaterBridge] = useState(false)
  const [desktopAutoUpdate, setDesktopAutoUpdate] = useState({
    visible: false,
    currentVersion: '',
    latestVersion: '',
    statusMessage: 'Checking for updates...',
    progressPercent: 0,
    isDownloading: false,
    isInstalling: false,
  })
  const mobileInitDoneRef = useRef(false)
  const splashHiddenRef = useRef(false)

  // Pages that don't require authentication
  const publicPages = ['/', '/app', '/download', '/clinic-login', '/register-clinic',
'/login', '/signup', '/upgrade', '/review-upgrade-request', '/super-admin-login',
'/privacy', '/privacy-policy', '/terms', '/refund-policy', '/contact', '/clinic-erp',
'/clinic-management-software-india', '/features', '/pricing', '/blog/[slug]']
  const isPublicPage = publicPages.includes(router.pathname)
  const superAdminOnlyPages = ['/super-admin', '/super-admin-profile']

  // Landing pages that should not have page transitions
  const landingPages = ['/', '/download', '/about', '/services', '/gallery', '/contact',
'/privacy-policy', '/terms', '/refund-policy', '/clinic-erp',
```

```
'/clinic-management-software-india', '/features', '/pricing', '/blog/[slug]']
  const isLandingPage = landingPages.includes(router.pathname)

  // Edit pages that use their own EditLayout (no need for main Layout wrapper)
  const editPages = ['/edit', '/edit-about', '/edit-services', '/edit-gallery',
'/edit-contact', '/super-admin-profile', '/clinic/branding-builder']
  const isEditPage = editPages.includes(router.pathname)
  const isBrandingBuilderPage = router.pathname === '/clinic/branding-builder'

  const isNativeCapacitorRuntime = () => {
    if (typeof window === 'undefined') return false
    const cap = (window as any).Capacitor
    if (!cap) return false
    if (typeof cap.isNativePlatform === 'function') return !!cap.isNativePlatform()
    if (typeof cap.getPlatform === 'function') return cap.getPlatform() !== 'web'
    return false
  }

  useEffect(() => {
    if (typeof window === 'undefined' || process.env.NODE_ENV !== 'development') return

    const w = window as any
    if (w.__devConsoleFiltered) return

    const originalLog = console.log
    const blockedPrefixes = ['[HMR] connected', '[Fast Refresh] rebuilding']

    console.log = (...args: any[]) => {
      const firstArg = typeof args[0] === 'string' ? args[0] : ''
      if (blockedPrefixes.some((prefix) => firstArg.startsWith(prefix))) {
        return
      }
      originalLog(...args)
    }

    w.__devConsoleFiltered = true
  }, [])

  useEffect(() => {
    if (typeof window === 'undefined') return
    const isNativeMobile = isNativeCapacitorRuntime()
    setIsNativeMobileApp(isNativeMobile)
  }, [])

  useEffect(() => {
    if (typeof window === 'undefined') return
    setHasDesktopUpdaterBridge(
      !!window.electronAPI?.checkForUpdates && !!window.electronAPI?.onUpdaterEvent
    )
  }, [])

  useEffect(() => {
    if (typeof window === 'undefined') return
    if (!window.electronAPI?.onUpdaterEvent) return

    const removeListener = window.electronAPI.onUpdaterEvent((payload) => {
      setDesktopAutoUpdate((prev) => {
        if (payload.event === 'checking-for-update') {
          return {
            ...prev,
            statusMessage: payload.status || 'Checking for updates...'
          }
```

```
        }

        if (payload.event === 'update-available') {
          return {
            ...prev,
            visible: true,
            currentVersion: versionCheck.currentVersion || prev.currentVersion,
            latestVersion: payload.version || prev.latestVersion,
            statusMessage: payload.status || 'Update available',
            progressPercent: 0,
            isDownloading: false,
            isInstalling: false,
          }
        }

        if (payload.event === 'download-progress') {
          return {
            ...prev,
            visible: true,
            statusMessage: payload.status || 'Downloading update...',
            progressPercent: typeof payload.percent === 'number' ? payload.percent :
prev.progressPercent,
            isDownloading: true,
          }
        }

        if (payload.event === 'update-downloaded') {
          return {
            ...prev,
            visible: true,
            statusMessage: payload.status || 'Update downloaded. Restarting...',
            progressPercent: 100,
            isDownloading: false,
            isInstalling: true,
          }
        }

        if (payload.event === 'error') {
          return {
            ...prev,
            visible: true,
            statusMessage: payload.error || payload.status || 'Update failed',
            isDownloading: false,
            isInstalling: false,
          }
        }

        return prev
      })
    })

    window.electronAPI.checkForUpdates?.().then((result) => {
      if (!result?.ok) {
        setDesktopAutoUpdate((prev) => ({
          ...prev,
          visible: true,
          statusMessage: result?.error || 'Unable to check for updates',
          isDownloading: false,
          isInstalling: false,
        }))
      }
    }).catch(() => {
```

```
      setDesktopAutoUpdate((prev) => ({
        ...prev,
        visible: true,
        statusMessage: 'Unable to check for updates',
        isDownloading: false,
        isInstalling: false,
      }))
    })

    return () => {
      removeListener?.()
    }
  }, [versionCheck.currentVersion])

  const startDesktopAutoUpdate = async () => {
    if (!window.electronAPI?.startUpdateDownload) return

    setDesktopAutoUpdate((prev) => ({
      ...prev,
      isDownloading: true,
      statusMessage: 'Starting download...',
    }))

    const result = await window.electronAPI.startUpdateDownload()
    if (!result.ok) {
      setDesktopAutoUpdate((prev) => ({
        ...prev,
        isDownloading: false,
        statusMessage: result.error || 'Unable to start update download',
      }))
    }
  }

  // Apply cached clinic branding immediately to avoid blocking route transitions on
network.
  useEffect(() => {
    // Reset to default branding for super admin pages and the public landing page
    if (
      router.pathname === '/super-admin' ||
      router.pathname === '/super-admin-login' ||
      router.pathname === '/super-admin-profile' ||
      router.pathname === '/register-clinic' ||
      router.pathname === '/review-upgrade-request' ||
      router.pathname === '/' ||
      router.pathname === '/clinic-erp' ||
      router.pathname === '/clinic-management-software-india' ||
      router.pathname === '/features' ||
      router.pathname === '/pricing' ||
      router.pathname === '/download' ||
      router.pathname === '/about' ||
      router.pathname === '/services' ||
      router.pathname === '/gallery' ||
        router.pathname === '/contact' ||
        router.pathname === '/privacy' ||
        router.pathname === '/privacy-policy' ||
        router.pathname === '/terms' ||
        router.pathname === '/refund-policy'
    ) {
      localStorage.removeItem('clinicName')
      localStorage.removeItem('clinicIcon')
        setPageTitle(SITE_TITLE)
      setFaviconUrl('/favicon.png')
```

```
      // Keep default blue — do NOT apply any clinic theme
      document.documentElement.style.setProperty('--brand', '#3B82F6')
      document.documentElement.setAttribute('data-theme', 'blue')
      return
    }

    const storedClinicName = localStorage.getItem('clinicName')
    const storedClinicIcon = localStorage.getItem('clinicIcon')

      setPageTitle(storedClinicName ? `${storedClinicName} ERP` : SITE_TITLE)
    setFaviconUrl(storedClinicIcon || '/favicon.png')

    // Apply cached theme — set data-theme FIRST so CSS variable overrides fire immediately
    const cachedTheme = localStorage.getItem('clinicTheme') || 'blue'
    document.documentElement.setAttribute('data-theme', cachedTheme)

    const cachedBrand = localStorage.getItem('clinicBrandColor')
    const cachedBrandDark = localStorage.getItem('clinicBrandColorDark')
    if (cachedBrand) {
      document.documentElement.style.setProperty('--brand', cachedBrand)
    }
    if (cachedBrandDark) {
      document.documentElement.style.setProperty('--brand-dark', cachedBrandDark)
    }
  }, [router.pathname])

  useEffect(() => {
    // Skip auth check for public pages
    if (isPublicPage) {
      setAuthChecked(true)
      return
    }

    let cancelled = false

    const applyClinicBranding = (clinic: any) => {
      if (!clinic) return

      const clinicName = clinic.name || 'Clinic'
      localStorage.setItem('clinicName', clinicName)
      setPageTitle(`${clinicName} ERP`)

      const icon = clinic.iconUrl || clinic.logoUrl || ''
      if (icon) {
        localStorage.setItem('clinicIcon', icon)
        setFaviconUrl(icon)
      }

      // Apply clinic theme gradient via CSS variable
      const themeGradient = clinic.themeGradient || 'blue'
      const themeMap: Record<string, { brand: string; brandDark: string }> = {
        blue:    { brand: '#3B82F6', brandDark: '#60A5FA' },
        purple:  { brand: '#8B5CF6', brandDark: '#A78BFA' },
        emerald: { brand: '#10B981', brandDark: '#34D399' },
        rose:    { brand: '#F43F5E', brandDark: '#FB7185' },
        teal:    { brand: '#22C55E', brandDark: '#4ADE80' },
      }
      const colors = themeMap[themeGradient] || themeMap.blue
      document.documentElement.style.setProperty('--brand', colors.brand)
      document.documentElement.style.setProperty('--brand-dark', colors.brandDark)
      // Set data-theme so CSS variable blocks in globals.css fire for ALL blue/sky/indigo
utilities
```

```
      document.documentElement.setAttribute('data-theme', themeGradient)
      localStorage.setItem('clinicTheme', themeGradient)
      localStorage.setItem('clinicBrandColor', colors.brand)
      localStorage.setItem('clinicBrandColorDark', colors.brandDark)
    }

    const handleUnauthenticated = (storedClinicId: string | null) => {
      if (superAdminOnlyPages.includes(router.pathname)) {
        router.push('/super-admin-login')
        return
      }

      const isNativeApp = typeof window !== 'undefined' && (!!(window as any).electronAPI ||
isNativeCapacitorRuntime())
      clearCachedCurrentUser()
      if (isNativeApp) {
        const next = storedClinicId
          ? `/login?clinicId=${encodeURIComponent(storedClinicId)}`
          : '/login'
        router.push(next)
      } else if (storedClinicId) {
        router.push(`/login?clinicId=${storedClinicId}`)
      } else {
        router.push('/clinic-login')
      }
    }

    const enforceRouteAccess = (user: any, storedClinicId: string | null) => {
      if (!user) {
        handleUnauthenticated(storedClinicId)
        return false
      }

      // Allow access if there's a stored clinic session even if user DB record doesn't have
clinicId.
      if (user.role !== 'super_admin' && !user.clinicId && !storedClinicId) {
        showError('You must log in with a clinic to access the ERP system')
        router.push('/clinic-login')
        return false
      }

      // Super admin can only access super-admin pages.
      const superAdminPages = superAdminOnlyPages
      if (user.role === 'super_admin' && !superAdminPages.includes(router.pathname)) {
        router.push('/super-admin')
        return false
      }

      // Non-super-admin users cannot access super-admin page.
      if (user.role !== 'super_admin' && superAdminPages.includes(router.pathname)) {
        showError('Access denied')
        router.push('/dashboard')
        return false
      }

      const clinicUpgradeRequired = Boolean(user?.clinic?.upgradeRequired)
      const isUpgradeRoute = router.pathname === '/upgrade'
      if (user.role !== 'super_admin' && clinicUpgradeRequired && !isUpgradeRoute) {
        router.push('/upgrade?reason=trial_expired')
        return false
      }
```

```
  return true
}

const storedClinicId = localStorage.getItem('clinicId')
const cachedUser = getCachedCurrentUser<any>()
if (!cachedUser) {
  // If offline with no cache, allow render immediately (offline banner will show)
  if (!navigator.onLine) {
    setAuthChecked(true)
  } else {
    setAuthChecked(false)
  }
}

// Fast path: use cached user to avoid blocking initial render on network.
if (cachedUser) {
  setActiveUser(cachedUser)
  if (enforceRouteAccess(cachedUser, storedClinicId)) {
    setAuthChecked(true)
    if (cachedUser.clinic) {
      applyClinicBranding(cachedUser.clinic)
      const subEnd = cachedUser.clinic.subscriptionEnd
      const subPlan = cachedUser.clinic.subscriptionPlan
      const isPro = subPlan === 'pro' && (!subEnd || new Date(subEnd) > new Date())
      if (isPro) { setIsProUser(true); localStorage.setItem('clinicIsPro', '1') }
    }
  }
}

// Revalidate auth in background to keep cache and clinic metadata fresh.
const checkAuth = async () => {
  // If offline, trust cached session entirely — do not attempt redirect
  if (!navigator.onLine) {
    if (!cachedUser) {
      // No cache at all while offline: show app but it will be limited
      setAuthChecked(true)
    }
    return
  }
  try {
    const res = await fetch('/api/auth/me', {
      method: 'GET',
      credentials: 'include',
      cache: 'no-store'
    })
    const data = await res.json()

    if (cancelled) return

    if (!data.user) {
      setActiveUser(null)
      handleUnauthenticated(storedClinicId)
      return
    }

    setActiveUser(data.user)
    setCachedCurrentUser(data.user)

    // Store clinic info - prioritize user's clinic, fallback to stored clinic.
    if (data.user.clinic) {
      localStorage.setItem('clinicId', data.user.clinic.clinicId)
      applyClinicBranding(data.user.clinic)
```

```
          const subEnd = data.user.clinic.subscriptionEnd
          const subPlan = data.user.clinic.subscriptionPlan
          const isPro = subPlan === 'pro' && (!subEnd || new Date(subEnd) > new Date())
          setIsProUser(isPro)
          localStorage.setItem('clinicIsPro', isPro ? '1' : '0')
        } else if (storedClinicId) {
          localStorage.setItem('clinicId', storedClinicId)
        }

        if (enforceRouteAccess(data.user, storedClinicId)) {
          setAuthChecked(true)
        }
      } catch (err) {
        if (cancelled) return
        // Network error (offline / server unreachable) — keep the cached session
        // so the user stays on their current page instead of being kicked to login.
        if (cachedUser) {
          setActiveUser(cachedUser)
          setAuthChecked(true)
        } else {
          setActiveUser(null)
          // No cache and cannot reach server — redirect to login only when a
          // stored clinicId exists (so the login URL is correct), otherwise
          // go to clinic-login as a last resort.
          handleUnauthenticated(storedClinicId)
        }
      }
    }

    checkAuth()

    return () => {
      cancelled = true
    }
  }, [router.pathname, isPublicPage, router])

  useEffect(() => {
    if (typeof window === 'undefined') return
    if (!activeUser?.clinic || activeUser?.role === 'super_admin') return
    if (router.pathname === '/upgrade') return

    const clinic = activeUser.clinic
    const isTrial = clinic.subscriptionStatus === 'trial'
    if (!isTrial) return

    const trialEndRaw = clinic.trialEndsAt
    const trialDaysTotal = Number(clinic.trialDaysTotal || 0)
    if (!trialEndRaw || !trialDaysTotal || trialDaysTotal <= 0) return

    const trialEnd = new Date(trialEndRaw)
    if (Number.isNaN(trialEnd.getTime())) return

    const now = new Date()
    if (now >= trialEnd) return

    const trialStart = new Date(trialEnd.getTime() - trialDaysTotal * 24 * 60 * 60 * 1000)
    const halfPoint = new Date(trialStart.getTime() + (trialEnd.getTime() -
trialStart.getTime()) / 2)
    if (now < halfPoint) return

    const reminderKey = `trialReminderLastSeen_${clinic.id || clinic.clinicId ||
'clinic'}_${activeUser.id || 'user'}`
```

```
    const lastSeenRaw = localStorage.getItem(reminderKey)
    const lastSeen = lastSeenRaw ? Number(lastSeenRaw) : 0
    const elapsed = now.getTime() - lastSeen
    const twentyFourHoursMs = 24 * 60 * 60 * 1000

    if (!lastSeen || elapsed >= twentyFourHoursMs) {
      const daysLeft = Number(clinic.trialDaysLeft || 0)
      setTrialReminderMessage(
        daysLeft > 0
          ? `Your free trial is in its final phase. ${daysLeft} day(s) left before upgrade
is required.`
          : 'Your free trial is about to end. Upgrade now to prevent service interruption.'
```

File: pages/api/process-bill.ts

```
            const numParts = nums.filter(n => n.isNum)
            const textParts = nums.filter(n => !n.isNum)

            if (textParts.length >= 1 && numParts.length >= 2) {
                const namePart = textParts.find(p => p.raw.length > 2 &&
!NON_ITEM_LINE.test(p.raw))
                if (namePart) {
                    // qty = first small number (<= 10000), rate = second, amount = last
                    const qtyC = numParts.find(n => n.n > 0 && n.n <= 10000)
                    const amtC = numParts[numParts.length - 1]
                    const rateC = numParts.find(n => n !== qtyC && n !== amtC && n.n > 0)
                    if (qtyC && amtC && qtyC.n > 0) {
                        const rate = rateC ? rateC.n : amtC.n / qtyC.n
                        items.push(makeBillItem(namePart.raw, qtyC.n, rate))
                        continue
                    }
                }
            }
        }

        // ── Strategy C: multi-line product block ──────────────────────────────
        // Line i:   "MEDICINE NAME 100ML"
        // Line i+1: "1"  (serial/line number)
        // Line i+2: "93.001860.0020"  (concatenated price+amount+qty)
        if (i + 2 < lines.length) {
            const nextLine = lines[i + 1]
            const numLine = lines[i + 2]

            if (nextLine.match(/^\d{1,3}$/) && /^[\d.]+$/.test(numLine)) {
                // Try to split concatenated number: PRICE.00AMOUNT.00QTY
                const m = numLine.match(/^([\d]+\.[\d]{2})([\d]+\.[\d]{2})(\d{1,4})$/)
                if (m) {
                    const rate = parseNum(m[1]), amount = parseNum(m[2]), qty =
parseInt(m[3])
                    if (qty > 0 && rate > 0) {
                        items.push(makeBillItem(line, qty, rate))
                        i += 2
                        continue
                    }
                }
            }
        }

        // ── Strategy D: space-separated flexible line ─────────────────────────
        // Handles any row that ends with 2–6 numbers after a product name.
        // Tolerates: serial, HSN, batch, expiry, MRP, discount columns.
        const normLine = line.replace(/\s+/g, ' ').trim()
        // Extract all tokens; find trailing numeric tokens
        const tokens = normLine.split(' ')
        const trailingNums: string[] = []
        for (let j = tokens.length - 1; j >= 0; j--) {
            if (/^[\d,]+\.?\d*$/.test(tokens[j])) {
                trailingNums.unshift(tokens[j])
            } else break
        }

        if (trailingNums.length >= 2) {
            const nameTokens = tokens.slice(0, tokens.length - trailingNums.length)
            // Strip leading serial number from name
            if (/^\d{1,3}$/.test(nameTokens[0])) nameTokens.shift()
```

```
            const name = nameTokens.join(' ').trim()

            if (name.length > 2 && !NON_ITEM_LINE.test(name)) {
                const nums = trailingNums.map(parseNum)
                // Last number is usually the total amount; the smallest integer-ish number
is qty
                const amount = nums[nums.length - 1]
                // Find qty: a whole number > 0 and <= 10000 that's not the last column
                let qty = 0, rate = 0
                for (let j = 0; j < nums.length - 1; j++) {
                    const n = nums[j]
                    if (Number.isInteger(n) && n > 0 && n <= 10000) { qty = n; break }
                }
                // If no clean integer found, try first number as qty
                if (qty === 0 && nums.length >= 3) qty = Math.round(nums[0])
                // Rate: column just after qty, or derive from amount/qty
                if (qty > 0) {
                    const qtyIdx = nums.findIndex(n => Math.round(n) === qty)
                    rate = nums[qtyIdx + 1] || 0
                    if (rate === 0 || rate === amount) rate = amount / qty
                    if (rate > 0 && qty * rate <= amount * 1.6) {
                        items.push(makeBillItem(name, qty, rate))
                        continue
                    }
                }
            }
        }

        // ── Strategy E: product-name-only line (qty on next line or unknown) ──
        // Last resort: if line looks like a medicine name and has no numbers,
        // peek at next 3 lines for quantity + price data.
        if (/^[A-Z][A-Z0-9 \-\/\.]+$/.test(line) && line.length > 4 && line.length < 60 && i
+ 1 < lines.length) {
            const peek = lines[i + 1]
            // Next line all-numbers → multi-line format with concatenated prices
            if (/^\d+$/.test(peek) && i + 2 < lines.length) {
                const dateLine = lines[i + 2]
                const m2 = dateLine.match(/^([\d]+\.[\d]{2})([\d]+\.[\d]{2})(\d{1,4})$/)
                if (m2) {
                    const rate = parseNum(m2[1]), qty = parseInt(m2[3])
                    if (qty > 0 && rate > 0) {
                        items.push(makeBillItem(line, qty, rate))
                        i += 2
                        continue
                    }
                }
            }
        }
    }

    // ── Deduplicate (Vision sometimes produces repeated page text) ────────────────
    const uniqueItems = dedupeItems(items)

    // ── Tax info completion
──────────────────────────────────────────────────────────────────────────────
    if (taxInfo.subtotal === 0 && uniqueItems.length > 0) {
        taxInfo.subtotal = uniqueItems.reduce((s, it) => s + it.price * it.qty, 0)
    }
    if (taxInfo.grandTotal === 0) {
        taxInfo.grandTotal = taxInfo.subtotal + taxInfo.cgst + taxInfo.sgst + taxInfo.igst
    }
    if (taxInfo.cgst > 0 && taxInfo.cgstPercent === 0 && taxInfo.subtotal > 0)
```

```
        taxInfo.cgstPercent = (taxInfo.cgst / taxInfo.subtotal) * 100
    if (taxInfo.sgst > 0 && taxInfo.sgstPercent === 0 && taxInfo.subtotal > 0)
        taxInfo.sgstPercent = (taxInfo.sgst / taxInfo.subtotal) * 100
    if (taxInfo.igst > 0 && taxInfo.igstPercent === 0 && taxInfo.subtotal > 0)
        taxInfo.igstPercent = (taxInfo.igst / taxInfo.subtotal) * 100
    if (taxInfo.cgst === 0 && taxInfo.sgst === 0 && taxInfo.igst === 0 &&
        taxInfo.grandTotal > taxInfo.subtotal && taxInfo.subtotal > 0) {
        const totalTax = taxInfo.grandTotal - taxInfo.subtotal
        taxInfo.cgst = totalTax / 2; taxInfo.sgst = totalTax / 2
        taxInfo.cgstPercent = (taxInfo.cgst / taxInfo.subtotal) * 100
        taxInfo.sgstPercent = (taxInfo.sgst / taxInfo.subtotal) * 100
    }

    return { items: uniqueItems, taxInfo }
}

// Configure Cloudinary
cloudinary.v2.config({
    cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
    api_key: process.env.CLOUDINARY_API_KEY,
    api_secret: process.env.CLOUDINARY_API_SECRET
})

export const config = {
    api: {
        bodyParser: false,
        externalResolver: true,
    },
}

export default async function handler(req: NextApiRequest, res: NextApiResponse) {
    // Log for debugging on Vercel

    // CRITICAL: Add authentication for multi-tenant isolation
    const user = await requireAuth(req, res)
    if (!user) return

    if (!isFeatureAllowed((user as any).clinic?.subscriptionPlan, 'upload_bill')) {
        return res.status(403).json({
            error: 'Upload Bill is available in Standard plan.',
            code: 'FEATURE_RESTRICTED',
            upgradeUrl: '/upgrade',
        })
    }

    if (req.method !== 'POST') {
        return res.status(405).json({ error: 'Method not allowed' })
    }

    let file: any = null

    try {
        const form = formidable({
            maxFileSize: 10 * 1024 * 1024, // 10MB
            keepExtensions: true,
        })

        let fields: any, files: any
        try {
            [fields, files] = await form.parse(req)
        } catch (parseError: any) {
            return res.status(400).json({
```

```
                error: 'Failed to parse upload',
                details: parseError.message
            })
        }

        file = Array.isArray(files.file) ? files.file[0] : files.file

        if (!file) {
            return res.status(400).json({ error: 'No file uploaded' })
        }


        // Upload to Cloudinary as raw file (not image) for proper PDF handling
        let billUrl = ''
        try {
            const result = await cloudinary.v2.uploader.upload(file.filepath, {
                folder: 'erp-flow-studios/bills',
                resource_type: 'raw', // Use 'raw' for PDFs instead of 'auto'
                type: 'upload',
                access_mode: 'public',
                overwrite: false
            })
            billUrl = result.secure_url
        } catch (uploadError: any) {
            throw new Error('Failed to upload bill to Cloudinary: ' + uploadError.message)
        }

        // Fetch all products from user's clinic to match with bill items (including tags
for better matching)
        const whereClause = await getClinicAwareDoctorFilter(user, prisma)
        const clinicId = getClinicIdFromUser(user)
        const allProducts = await prisma.product.findMany({
            where: whereClause,
            select: {
                id: true,
                name: true,
                purchasePriceRupees: true,
                purchasePricePerUnit: true,
                unit: true,
                tags: true
            }
        })

        // Fetch existing product mappings for automatic matching
        const existingMappings = await prisma.billProductMapping.findMany({
            where: {
                product: whereClause,
                billProductName: {
                    startsWith: `${clinicId}::`
                }
            },
            select: {
                billProductName: true,
                mappedProductId: true
            }
        })
        const mappingDict = new Map(existingMappings.map((m: any) => {
            const scopedName = m.billProductName || ''
            const unscopedName = scopedName.includes('::') ?
scopedName.split('::').slice(1).join('::') : scopedName
            return [unscopedName, m.mappedProductId]
        }))
```

```
        // ─── Determine OCR + AI pipeline based on subscription plan ──────────────
        //
        //  PRO/BASIC+AI_OCR : Vision OCR  -> Gemini AI parsing
        //  BASIC            : Tesseract   -> Gemini AI parsing
        //
        const clinicPlan: string = (user as any).clinic?.subscriptionPlan || 'basic'
        const hasVisionAccess = clinicPlan === 'pro' || clinicPlan === 'basic_ai_ocr' ||
clinicPlan === 'standard_ai_ocr'

        // Resolve requested provider from frontend field, then clinic setting, then default
        const requestedProvider = Array.isArray(fields.ocrProvider) ? fields.ocrProvider[0]
: fields.ocrProvider
        const rawProvider: string = requestedProvider || (user as any).clinic?.ocrProvider
|| 'tesseract'

        // Enforce plan gate for Vision OCR access.
        // Applies to BOTH the frontend-requested value AND the stored clinic value.
        const ocrProvider: string = (rawProvider === 'google_vision' && !hasVisionAccess) ?
'tesseract' : rawProvider

        const googleVisionKey: string | null = (user as any).clinic?.apiGoogleVisionKey ||
null
        const usingGoogleVision = ocrProvider === 'google_vision' && (!!googleVisionKey ||
(!!process.env.GOOGLE_CLIENT_EMAIL && !!process.env.GOOGLE_PRIVATE_KEY))
        const parsePipeline = usingGoogleVision ? 'vision+gemini' : 'tesseract+gemini'

        // Gemini AI parser runs for ALL plans — it just receives whatever text OCR
produced.
        // Pro: Gemini gets high-quality Vision text.
        // Basic: Gemini gets Tesseract text (lower quality but still useful).
        const useGeminiAI = true

        console.log(`[process-bill] plan=${clinicPlan} ocr=${ocrProvider}
gemini=${useGeminiAI} pipeline=${parsePipeline}`)

        // ─── Check Vision usage limit before calling the API ────────────────────
        if (usingGoogleVision) {
            try {
                await checkVisionLimit()
            } catch (limitError: any) {
                if (limitError.code === 'VISION_LIMIT_EXCEEDED') {
                    return res.status(429).json({ error: limitError.message, code:
'VISION_LIMIT_EXCEEDED' })
                }
                throw limitError
            }
        }

        // ─── Extract text from the uploaded file ────────────────────────────────
        let billItems: Array<{ name: string; qty: number; price: number; bottleQuantity:
number; unitsPerBottle: number }> = []
        let taxInfo: TaxInfo = {
            subtotal: 0, cgst: 0, sgst: 0, igst: 0,
            cgstPercent: 0, sgstPercent: 0, igstPercent: 0,
            grandTotal: 0, taxLocked: false
        }
        let rawExtractedText = ''

        if (file.mimetype === 'application/pdf') {
            // ── Step 1: try text extraction via pdf-parse ──────────────────────
            try {
```

```
                const pdfParse = require('pdf-parse')
                const dataBuffer = fs.readFileSync(file.filepath)
                const pdfData = await pdfParse(dataBuffer)
                rawExtractedText = pdfData.text || ''

                if (rawExtractedText.trim().length > 20) {
                    const parseResult = parseBillText(rawExtractedText)
                    billItems = parseResult.items
                    taxInfo = parseResult.taxInfo
                    console.log(`[process-bill] pdf-parse extracted
${rawExtractedText.length} chars, found ${billItems.length} items`)
                }
            } catch (pdfError: any) {
                console.warn('[process-bill] pdf-parse failed:', pdfError.message)
            }

            // ── Step 2: OCR fallback when pdf-parse yields no items ──────────────
            // (scanned/image-based PDFs have no embedded text)
            if (billItems.length === 0) {
                console.warn('[process-bill] No items from pdf-parse; attempting OCR on
PDF...')

                if (ocrProvider === 'google_vision') {
                    // ── Google Vision: send PDF bytes directly (no image conversion
needed) ──
                    const credential = googleVisionKey || null
                    const hasEnvCreds = !!process.env.GOOGLE_CLIENT_EMAIL &&
!!process.env.GOOGLE_PRIVATE_KEY

                    if (credential || hasEnvCreds) {
                        try {
                            const pdfBuffer = fs.readFileSync(file.filepath)

                            let ocrText = ''
                            if (credential) {
                                ocrText = await extractTextGoogleVisionPdf(pdfBuffer,
credential)
                            } else {
                                // Use environment variables for Vercel serverless
compatibility
                                const vision = require('@google-cloud/vision')
                                const client = new vision.ImageAnnotatorClient({
                                    credentials: {
                                        client_email: process.env.GOOGLE_CLIENT_EMAIL,
                                        private_key:
process.env.GOOGLE_PRIVATE_KEY?.replace(/\\n/g, '\n')
                                    },
                                    projectId: process.env.GOOGLE_PROJECT_ID
                                })
                                const [result] = await client.batchAnnotateFiles({
                                    requests: [{
                                        inputConfig: { content: pdfBuffer, mimeType:
'application/pdf' },
                                        features: [{ type: 'DOCUMENT_TEXT_DETECTION' }],
                                        pages: [1, 2, 3, 4, 5],
                                    }]
                                })
                                const pages: any[] = (result as
any).responses?.[0]?.responses || []
                                ocrText = pages.map((p: any) => p.fullTextAnnotation?.text
|| '').join('\n')
                            }
```

```
                            rawExtractedText = ocrText
                            console.log(`[process-bill] Google Vision PDF OCR extracted
${ocrText.length} chars`)
                            await incrementVisionUsage()

                            if (ocrText.trim().length > 20) {
                                const ocrResult = parseBillText(ocrText)
                                billItems = ocrResult.items
                                taxInfo = ocrResult.taxInfo
                            }
                        } catch (ocrError: any) {
                            console.warn('[process-bill] Google Vision PDF OCR failed:',
ocrError.message)
                        }
                    } else {
                        console.warn('[process-bill] google_vision selected but no
credentials configured')
                    }
                } else {
                    // ── Tesseract path: try sharp (needs poppler), otherwise inform user
──
                    const imgBuffer = await pdfToImageBuffer(file.filepath)
                    if (imgBuffer) {
                        try {
                            const ocrText = await performOCR(imgBuffer, ocrProvider,
googleVisionKey)
                            rawExtractedText = ocrText
                            console.log(`[process-bill] Tesseract PDF OCR extracted
${ocrText.length} chars`)
                            if (ocrText.trim().length > 20) {
                                const ocrResult = parseBillText(ocrText)
                                billItems = ocrResult.items
                                taxInfo = ocrResult.taxInfo
                            }
                        } catch (ocrError: any) {
                            console.warn('[process-bill] Tesseract OCR failed:',
ocrError.message)
                        }
                    } else {
                        // Both sharp+poppler and pdfjs-dist rendering failed
                        rawExtractedText = '__IMAGE_PDF_NO_POPPLER__'
                        console.warn('[process-bill] Scanned PDF detected but all
PDF-to-image renderers failed')
                    }
                }
            }
        } else {
            // ── Image file: OCR directly
──────────────────────────────────────────────

            try {
                const imageBuffer = fs.readFileSync(file.filepath)
                const ocrText = await performOCR(imageBuffer, ocrProvider, googleVisionKey)
                rawExtractedText = ocrText
                console.log(`[process-bill] Image OCR (${ocrProvider}) extracted
${ocrText.length} chars`)
                if (usingGoogleVision) await incrementVisionUsage()

                if (!ocrText || ocrText.trim().length === 0) {
                    throw new Error('No text could be extracted from the image. Please
ensure the image is clear and well-lit.')
                }
```

```
                const parseResult = parseBillText(ocrText)
                billItems = parseResult.items
                taxInfo = parseResult.taxInfo
            } catch (ocrError: any) {
                if (ocrError.message?.includes('No text could be extracted')) throw ocrError
                throw new Error(`Failed to process image (${ocrProvider}):
${ocrError.message}`)
            }
        }

        // ─── AI Bill Parsing (Gemini)
──────────────────────────────────────────────────────────────────────────────
        // Runs for ALL plans after OCR text is available.
        //   Pro   → Vision text  → Gemini (high accuracy)
        //   Basic → Tesseract text → Gemini (improved vs regex alone)
        // Falls back to regex-parsed billItems if AI fails or returns nothing.
        let aiParseUsed = false
        let aiParseModel = ''
        let aiParseCached = false
        const regexItemCountBeforeAI = billItems.length
        if (useGeminiAI && rawExtractedText && rawExtractedText.trim().length > 20 &&
            rawExtractedText !== '__IMAGE_PDF_NO_POPPLER__') {
            try {
                const aiResult = await parseBillWithAI(rawExtractedText)
                if (aiResult.items.length > 0) {
                    const aiCount = aiResult.items.length
                    const regexCount = regexItemCountBeforeAI
                    const aiLooksTooSmall = regexCount >= 5 && aiCount <= Math.max(2,
Math.floor(regexCount * 0.35))

                    if (aiLooksTooSmall) {
                        console.warn(
                            `[process-bill] AI parsed ${aiCount} items, but regex already
had ${regexCount}; keeping regex results`
                        )
                    } else {
                    // Convert AI items to the BillItem format used by the rest of the
pipeline.
                    // unitsPerBottle defaults to 1 when not specified.
                    billItems = aiResult.items
                        .filter(ai => ai.productName)
                        .map(ai => {
                            const upb = ai.unitsPerBottle ?? 1
                            const bottleQty = ai.bottleQuantity ?? (ai.quantity ?
ai.quantity / upb : 1)
                            const totalQty = ai.quantity ?? (bottleQty * upb)
                            const unitPriceVal = ai.unitPrice ?? 0
                            return {
                                name: ai.productName as string,
                                qty: totalQty,
                                price: unitPriceVal,
                                bottleQuantity: bottleQty,
                                unitsPerBottle: upb,
                            }
                        })
                    aiParseUsed = true
                    aiParseModel = aiResult.model
                    aiParseCached = aiResult.cached === true
                    console.log(`[process-bill] AI parsed ${billItems.length} items via
${aiResult.model}${aiParseCached ? ' (cache hit)' : ''}`)
                    }
```

```
                } else {
                    console.log('[process-bill] AI returned 0 items; using regex-parsed
billItems')
                }
            } catch (aiErr: any) {
                console.warn('[process-bill] AI parsing failed (using regex fallback):',
aiErr.message)
            }
        }

        // Match bill items with database products using keyword-based fuzzy matching
        const extractedData = billItems.map((billItem: { name: string; qty: number; price:
number; bottleQuantity: number; unitsPerBottle: number }) => {
            // First check if there's an existing mapping for this exact bill product name
            if (mappingDict.has(billItem.name)) {
                const mappedProductId = mappingDict.get(billItem.name)
                const mappedProduct = allProducts.find((p: any) => p.id === mappedProductId)

                if (mappedProduct) {
                    return {
                        productId: mappedProduct.id,
                        productName: billItem.name,
                        quantity: billItem.qty,
                        unitPrice: billItem.price,
                        bottleQuantity: billItem.bottleQuantity,
                        unitsPerBottle: billItem.unitsPerBottle,
                        matched: true,
                        matchedProductName: mappedProduct.name,
                        matchScore: 999, // High score for previously mapped items
                        requiresUserAction: false,
                        usedMapping: true
                    }
                }
            }

            // Extract significant keywords from bill item name (ignore common words,
numbers at end)
            const extractKeywords = (name: string): string[] => {
                return name.toLowerCase()
                    .replace(/\d+ml|\d+mg|\s+\d+$/gi, '') // Remove measurements and
trailing numbers
                    .replace(/[()]/g, ' ') // Remove parentheses
                    .split(/\s+/)
                    .filter(word => word.length > 2 && ['the', 'and', 'pet', 'bottal',
'pcs'].includes(word))
            }

            const billKeywords = extractKeywords(billItem.name)

            // Try to find matching product with keyword scoring
            let bestMatch = null
            let bestScore = 0

            for (const product of allProducts) {
                // First check if bill name matches any product tags (exact match)
                if (product.tags && product.tags.includes(billItem.name)) {
                    bestMatch = product
                    bestScore = 1000 // Highest score for tag match
                    break
                }

                const productKeywords = extractKeywords(product.name)
```

```
                // Calculate match score based on common keywords
                let score = 0
                for (const billWord of billKeywords) {
                    for (const prodWord of productKeywords) {
                        if (prodWord.includes(billWord) || billWord.includes(prodWord)) {
                            score += 1
                        }
                    }
                }

                // Bonus for exact substring match
                const productNameLower = product.name.toLowerCase()
                const billNameLower = billItem.name.toLowerCase()
                if (productNameLower.includes(billNameLower) ||
billNameLower.includes(productNameLower)) {
                    score += 2
                }

                if (score > bestScore) {
                    bestScore = score
                    bestMatch = product
                }
            }

            // Only accept matches with score >= 2 (more strict to reduce false positives)
            const matchedProduct = bestScore >= 2 ? bestMatch : null

            return {
                productId: matchedProduct?.id || null,
                productName: billItem.name,
                quantity: billItem.qty, // Total units to add to inventory (bottles × units
per bottle)
                unitPrice: billItem.price, // Price per unit (per ML/MG/etc)
                bottleQuantity: billItem.bottleQuantity, // Number of bottles purchased
                unitsPerBottle: billItem.unitsPerBottle, // Units per bottle (e.g., 100ML)
                matched: !!matchedProduct,
                matchedProductName: matchedProduct?.name || null,
                matchScore: bestScore,
                requiresUserAction: !matchedProduct // Flag for items needing user input
            }
        })

        // Separate matched and unmatched items
        const validItems = extractedData.filter(item => item.productId !== null)
        const unmatchedItems = extractedData.filter(item => item.productId === null)
```